

Personal Wealth Operating System

Master build brief, data model and Claude Code prompt library
Compressed from the 40-volume specification into a buildable plan

Version 1.0 | Single-user build | Next.js, Supabase, PostgreSQL

Companion files: `CLAUDE.md` (repo rules) and `0001_init.sql` (migration, phases 0-3).

1. How to use this document

This replaces the 40 volumes as the working brief. The volumes describe what PWOS should eventually be; this describes what gets built, in what order, by whom, starting Monday.

Nothing here contradicts the volumes. It cuts them down to the subset that a single developer with a part-time job and a degree to finish can actually ship.

The workflow with Claude Code is:

1. Create the repo. Drop `CLAUDE.md` at the root and `0001_init.sql` into `supabase/migrations/`.
2. Open Claude Code in the repo and paste the **Standing Brief** in section 3. Do this once per session, or leave it in `CLAUDE.md` and skip it.

3. Paste the **phase prompt** for whichever phase you are on. One phase per session.
 4. Deploy at the end of every phase. Use the app for a week before starting the next one.
- Do not paste all phases at once. The single most common failure mode for a project this size is a half-finished phase 6 sitting on top of a phase 2 that was never used.

Why not 200 volumes

You asked whether more specification would produce a better platform. It would not, and the reason is specific rather than a matter of taste.

Specification is only worth writing where it removes ambiguity for someone who is not in the room. Volumes 41-200 as scoped would describe screens for features whose value you cannot yet judge, for a user base of one whose preferences will change as soon as the first version is in daily use. Every page of it would be written before a single real transaction had been entered, which means it would be guessing. Meanwhile the specification itself becomes a second codebase to maintain: change your mind about the ledger and you now have forty documents that are quietly wrong.

The 40 volumes you have are enough. What was missing is a schema, a build order, and a definition of done. That is what follows.

2. Ground truth

Write this down because every later decision refers back to it.

The user

One person. UK-based computer science student, final year, graduating June 2027. Runs a small web design and marketing agency alongside the degree. Works paid shifts. Holds a small Trading 212 portfolio. Currently carrying an overdraft. Phone is the primary device; the laptop is for coursework and building.

What the app is for

Three jobs, in order of how often they happen:

1. **Before spending money**, answer "can I afford this and what does it cost me." Seconds, on a phone, one-handed.
 2. **Weekly**, answer "did last week go the way I planned." Two minutes.
 3. **Monthly**, answer "what changed, and what is the next thing to fix." Ten minutes.
- Everything in the 40 volumes that does not serve one of those three eventually, is deferred.

What it is not

Not multi-user. Not a product to sell. Not a Notion or Obsidian competitor - those are mature tools built by funded teams, and treating them as the benchmark is how this project dies. Not an advice engine: it describes and calculates, and stops there.

Constraints that shape the architecture

- One developer, part-time, with exams.
- Free-tier hosting for the foreseeable future. Supabase free tier pauses inactive projects; the app must survive a cold start gracefully.
- No paid market-data feed. Prices are entered manually or imported from broker CSV.
- No bank API. Open Banking aggregators cost money and require FCA-regulated intermediaries. Transactions are entered manually or imported from CSV, and the manual path must be fast enough that it is genuinely used.

That last point deserves emphasis. The volumes assume integrations that are not available to you. A tracker that depends on an import you cannot perform is a tracker you will abandon in week two. Quick manual entry is the core feature, not the fallback.

3. The standing brief

Paste this at the start of a Claude Code session, or keep it in CLAUDE.md.

You are the sole engineer on PWOS, a single-user personal finance and life dashboard for a UK student. Read CLAUDE.md before doing anything.

Hard rules:

- Money is BIGINT minor units. Never floats. Currency is a separate column.
- The transactions ledger is append-only. Corrections are new rows.
- Balances and totals are derived from the ledger, never stored.
- Every table has user_id and RLS with auth.uid() = user_id on all operations.
- Server-side auth on every route. Never trust a client-supplied user_id.
- The UI must distinguish recorded facts, calculated values, and AI output.
- No investment advice, no expected-return figures, no guarantees.

Working method:

- One phase at a time. State the plan, list the files you will touch, then build.
- Business logic in features/<name>/service.ts. Components never query the DB.
- Every feature ships with loading, empty and error states, mobile-first at 375px.
- Write the unit tests for calculations as you write the calculations.
- When a requirement is ambiguous, pick the simpler option and write an ADR.
- Do not add dependencies without saying what they replace.

Stop and ask before: changing the schema of an existing table with data in it, adding a paid third-party service, or starting a phase that has not been requested.

4. Data model

0001_init.sql implements phases 0-3. The design decisions worth defending:

One ledger, not two

The volumes specify income_transactions and expense_transactions as separate tables.

Use a single transactions table with a direction enum instead. Cash flow, budgets, net worth and every report need both sides; two tables means every query is a UNION and every bug has to be fixed twice. The direction column costs nothing and the indexes are the same.

Append-only ledger

amount_minor, occurred_on, direction and account_id are immutable after insert, enforced by a trigger. A mistake is voided and re-entered with corrects_id pointing at the original. This is what makes the historical reports trustworthy: a number that was right in March stays right in March.

Balances are a view

account_balances computes balance and overdraft headroom from opening_balance_minor plus the ledger. There is no current_balance column to drift out of sync. If this view is ever slow - it will not be, at your transaction volume, for years - convert it to a materialised view refreshed on write, and record the reason in an ADR.

Tables by phase

- **Phase 0-1:** profiles, accounts, categories, transactions
- **Phase 2:** recurring_rules, budgets
- **Phase 3:** goals, goal_contributions
- **Phase 4:** portfolios, instruments, investment_transactions, dividends, import_batches
- **Phase 5:** assets, liabilities, net_worth_snapshots
- **Phase 6:** clients, projects, invoices, invoice_lines
- **Phase 7:** modules, assignments, study_sessions
- **Phase 8:** ai_runs, ai_messages
- **Phase 9:** notes, resources, note_links

Investment tables, when you get there

```
instruments          (id, user_id, ticker, name, exchange, currency,
                      asset_type, sector, is_listed, notes)
investment_transactions(id, user_id, portfolio_id, instrument_id, kind,
                      units NUMERIC(20,8), price_minor, fee_minor,
                      fx_rate NUMERIC(18,8), occurred_on, external_id,
                      import_id, corrects_id)
holdings              -- VIEW: units and cost basis rolled up per instrument
dividends             (id, user_id, instrument_id, amount_minor,
                      withholding_minor, paid_on, external_id)
```

Units are `NUMERIC(20, 8)`, not integers - Trading 212 sells fractional shares and rounding them destroys the cost basis. Prices are minor units like everything else. `is_listed` exists so the app can flag an instrument that is not publicly traded rather than silently treating it as a normal holding.

5. Calculations

Define these once, test them once, and never inline them. The tests are the contract.

Cash position

```
headroom      = balance + overdraftLimit      (balance negative when overdrawn)
dailyBurn     = mean(daily 'out' total over trailing 90 days)
runwayDays    = floor(headroom / dailyBurn)
monthlySurplus= income - fixedCosts - subscriptions - variableBudget
```

Fixed costs and subscriptions come from categories flagged `is_fixed`. Variable budget is the user's own figure, not a calculated one.

Debt clearance

```
monthsToClear = ceil(overdraftUsed / monthlySurplus)  when surplus > 0, else null
```

Return null, and say so in the UI, when the surplus is zero or negative. Do not render a date built on a negative surplus - it will be wrong in a way that flatters.

The ladder

The allocation engine is the one piece of real logic in the finance module. Given a monthly surplus and goals ordered by priority, allocate the entire surplus to the first unfunded goal, then the next, and return a projected completion month for each.

```
function project(surplusMinor, goals) {
  if (surplusMinor <= 0) return goals.map(g => ({...g, months: null}))
  let cumulative = 0
  return goals.map(g => {
    const need = Math.max(0, g.targetMinor - g.savedMinor)
    const months = need === 0 ? 0 : Math.ceil(need / surplusMinor)
    cumulative += months
    return { ...g, need, months, completesInMonths: cumulative }
  })
}
```

Test cases: zero surplus, negative surplus, a goal already funded, a goal needing exactly one month, and rounding at 1 pence over a boundary.

Savings rate and net worth

```
savingsRate = income === 0 ? null : (income - totalOut) / income
netWorth    = sum(assets) + sum(accountBalances) - sum(liabilities)
```

Portfolio

```
costBasis = sum(buy units x price + fees) - sum(sold units x average cost)
value     = units x lastKnownPrice           -- price is user-entered, timestamped
unrealised= value - costBasis
```

Show the price timestamp next to any value derived from it. Do not compute annualised returns, IRR, or projections. If you later want time-weighted return, it needs daily valuation snapshots, which needs a price feed you do not have.

6. Build phases

Each phase is one Claude Code session, or a few. Deploy at the end of each. Use it before moving on.

Phase 0 - Foundation

Next.js with TypeScript strict, Tailwind, Supabase client wiring, email auth with a single allowed address, protected layout, mobile bottom nav, design tokens, `lib/money.ts` with tests, migration 0001 applied, deployed to Vercel.

Done when: you can sign in on your phone and see an empty dashboard at your own domain.

Build Phase 0 of PWOS. Scaffold the Next.js App Router project with TypeScript strict, Tailwind, and Supabase auth (email magic link, single permitted address from an env var). Apply `supabase/migrations/0001_init.sql`. Create the protected app shell: mobile bottom navigation with Position, Money, Ladder, Portfolio, More; a design token layer in `globals.css`; and `lib/money.ts` implementing parse, format and arithmetic on minor units with full unit tests. Add a Playwright test for sign-in. No features yet. Then give me the deploy steps for Vercel and Supabase.

Phase 1 - The ledger

Accounts CRUD. Categories with the seeded defaults. Transaction quick-add: amount, category chips, optional note, defaults to today and the main account, three taps total. Transaction list with month grouping, filter by category, void and correct. Derived balance everywhere.

Done when: you have logged a week of real spending on your phone without it being annoying. This is the phase that decides whether the project survives, so tune the quick-add until it takes under five seconds.

Build Phase 1 of PWOS: the ledger. Accounts CRUD (name, institution, kind, opening balance, overdraft limit). Category management seeded via `seed_default_categories`. A quick-add transaction sheet optimised for one-handed phone use: numeric keypad first, category as tappable chips ordered by recent use, note optional, date defaulting to today, submit in three taps. A transaction list grouped by month with category filter, and void-and-correct rather than edit. All balances read from the `account_balances` view. Include an RLS test that proves a second user sees zero rows. Mobile-first at 375px.

Phase 2 - Recurring, budgets and the Position screen

Recurring rules that materialise forecast rows without writing to the ledger until they actually happen. Monthly budget per category. The Position screen: current balance as the hero figure, headroom meter, monthly flow strip, the affordability check, and the clearance date when a surplus exists.

Done when: the Position screen tells you something you did not already know.

Build Phase 2 of PWOS. Recurring rules (weekly to yearly) that generate a forecast for the current and next month without inserting ledger rows until the date passes, plus a one-tap confirm to convert a forecast into a real transaction. Monthly budgets per category with actual-vs-target. Then the Position screen: hero balance, overdraft headroom meter, income/fixed/budget/surplus strip, an affordability checker that shows resulting headroom and runway

days, and a clearance projection using the calculations in the build brief - returning and displaying null when surplus is not positive. Unit-test every calculation against the worked examples.

Phase 3 - Goals and the ladder

Goals with target, priority and contributions. The ladder view: overdraft first, then goals

in priority order, each with a projected completion month. Progress meters.

Build Phase 3 of PWOS: goals and the allocation ladder. Goals CRUD with target, priority ordering (drag to reorder), and a contributions log. A ladder view that places overdraft clearance first, then active goals by priority, running the `project()` allocation function from the build brief and showing projected completion months. Show remaining amount per goal and a progress meter. Handle the zero-and-negative-surplus cases explicitly in the UI.

Phase 4 - Investments and Trading 212 import

Portfolios, instruments, investment transactions, dividends. CSV import with a real

pipeline: parse, map columns, validate, detect duplicates on `external_id`, preview, confirm, report. Holdings view with cost basis and unrealised P/L. Concentration display by sector.

Build Phase 4 of PWOS: the investment module. Schema migration for instruments, `investment_transactions` (units `NUMERIC(20,8)`), dividends and holdings as a view. A Trading 212 CSV importer with the full pipeline: upload, column auto-mapping with manual override, row validation, duplicate detection on `external_id`, preview table showing what will be inserted and skipped, confirm, and an import report written to `import_batches`. Holdings list with units, cost basis, current value from a user-entered timestamped price, and unrealised P/L. A concentration panel showing weight by sector and by instrument. Flag any instrument with `is_listed` false. No expected returns, no projections anywhere.

Phase 5 - Net worth and reviews

Assets, liabilities, monthly snapshots. The weekly and monthly review pages built from

recorded data only: what came in, what went out, biggest categories, goal movement, portfolio change.

Phase 6 - Business

Clients, projects, invoices with line items and status, business income and expenses tagged

separately from personal but rolling into the same net worth. Built for the agency you actually run, not a generic CRM.

Phase 7 - University

Modules, assignments with deadlines and weightings, study sessions. Deadline surfacing on the dashboard. Keep it small: the calendar app on your phone already works.

Phase 8 - AI assistant

One assistant, not ten personas. The personas in Volume 35 are a prompt-library idea, not an architecture - ten system prompts against the same model with the same data is one assistant with a topic switch, and building it as ten means ten things to maintain.

Give it read-only tools over your own data: `getPosition`, `getSpendByCategory`,

getGoalLadder, getHoldings, getUpcomingDeadlines. It answers from tool results, cites which numbers it used, and refuses to answer from memory. Every run is logged to ai_runs with the tools called and the response, so you can audit what it told you.

Build Phase 8 of PWOS: the assistant. A chat route calling the Anthropic API with tool use. Implement read-only tools over the user's own data: getPosition, getSpendByCategory(month), getGoalLadder, getHoldings, getUpcomingDeadlines. The system prompt must require answering only from tool results, stating which figures came from which tool, refusing to give investment advice or predict returns, and saying plainly when the data does not answer the question. Log every run and tool call to ai_runs. Render AI output in a visually distinct container labelled as generated. Rate-limit to a sane number of calls per day.

Phase 9 - Knowledge

Notes with tags and links, plus a resource library of URLs and your own summaries. Store metadata and your notes, never copies of copyrighted text. Defer the knowledge graph and semantic search until you have more than a hundred notes; before that they solve nothing.

7. What was cut, and when to add it back

Cutting these is the point of the document. Each has a trigger for reconsidering it.

- **Multi-user, workspaces, roles, RBAC** (Volume 39). Add when a second real person asks for an account. The `user_id` column and RLS on every table already make this possible without a redesign, which is the only part that had to be decided early.
- **Plugin SDK, marketplace, public API, webhooks** (Volume 32). Add when a third party wants to build against it. Until then the API surface is your own frontend.
- **Ten AI personas** (Volume 35). Collapsed to one assistant with tools. Add a second system prompt when one prompt is measurably failing at a specific task.
- **Knowledge graph, semantic search, spaced repetition** (Volume 11). Add at 100+ notes.
- **Automation and workflow builder** (Volume 30). A workflow builder is a programming language with a worse UI. Write the two or three cron jobs you actually need.
- **Multi-region, containers, IaC, observability stack** (Volume 31). Vercel and Supabase give you deploys, logs and backups. Add when you have paying users or an SLA.
- **Bank integrations**. Add if you ever want to pay an Open Banking aggregator. The import pipeline from Phase 4 is the seam it plugs into.
- **OCR receipts, voice, wearables, mobile apps**. These are Volume 40 wish-list items. A well-built responsive web app installed to the home screen covers the mobile case.

8. Definition of done, per phase

Before a phase counts as finished:

- Works at 375px width, one-handed, with a numeric keypad where numbers are entered.
- Loading, empty and error states exist and say something useful.
- Server-side validation on every mutation; the client is not the guard.
- An RLS test signs in as a second user and receives zero rows from every new table.
- Service-layer calculations have unit tests including the boundary cases named in

section 5.

- The migration runs clean on an empty database, forward-only.
- One ADR per non-obvious decision, in docs/adr/.
- Deployed, and used for at least a few days before the next phase begins.

9. The honest part

The app cannot make you wealthy, and no amount of specification changes that. What it can do is remove the excuse of not knowing where you stand, make the cost of a decision visible before you make it rather than after, and keep an honest record you can look back on.

The financial outcome will be decided by things outside the software: what you earn, what you spend, how long you hold, and whether you keep your commitments to yourself. The app's only job is to make those things visible enough that you cannot pretend otherwise.

Build Phase 0 and Phase 1. Use them for a month. Then decide what phase 2 should really be - by then you will know something that no specification written today could tell you.

Nothing in this document is financial advice.